ATTENDAI

Smart Employee Attendance System

HCL Internship Project — Technical Documentation

Version 2.0.0 | April 2026

AI-powered | Face Recognition | Fully On-Device | Anti-Spoofing | HR Portal

1. Project Overview

1.1 Problem Statement

Create an AI-driven attendance system that uses face recognition to automatically record student or employee attendance need to scan the latop and identify it belongs to the user. The system should recognize faces in real time from a live camera feed, mark attendance, and generate reports, ensuring accurate and efficient tracking

1.2 Our Solution

A deep learning-based face recognition system that:

- Recognizes employees through the laptop webcam in real-time
- Automatically records check-in and check-out times
- Calculates total work hours per employee per day
- Generates attendance reports with charts for HR management
- Detects liveness (real face vs printed photo) to prevent spoofing

Key Capabilities

Capability	Technology	Notes
Face Recognition	DeepFace + Facenet	128-d embeddings, cosine similarity
Anti-Spoofing	Laplacian + frame variation	Blocks phone/printed photos
Duplicate Guard	Cosine threshold 0.40	Prevents dual registration
Live Dashboard	Vanilla JS + Django	Real-time stats & charts
Reports	Filterable table	CSV export, HR-only
HR Auth	Django session auth	Login required for reports
100% On-Device	SQLite	No cloud, no data leakage

2. Technology Stack

Layer	Technology	Details
Backend API	FastAPI 0.110 + Uvicorn 0.29	Async REST API, CORS, automatic Swagger UI at /docs
Face Recognition	DeepFace 0.0.91 (Facenet)	128-dimensional FaceNet embeddings, cosine distance matching
Face Detection	MTCNN + OpenCV fallback	5-tier fallback strategy for maximum compatibility
Image Preprocessing	OpenCV 4.9 + CLAHE	Contrast enhancement, upscale/downscale normalisation
Database ORM	SQLAlchemy 2.0	Declarative models, session factory, dependency injection
Database Storage	SQLite	Two separate databases: attendance.db and hr_auth.db
Data Validation	Pydantic 2.6	Request/response schemas with strict type checking
Frontend Framework	Django 5.0	Template rendering, session management, CSRF protection
UI Layer	Vanilla JS + CSS variables	No external JS framework, dark theme, toast notifications
Anti-Spoofing	Laplacian variance + frame std	Dual-method liveness detection calibrated for webcam use
HTTP Communication	Fetch API (CORS)	Frontend JS calls FastAPI REST endpoints directly
Face Storage	File system (face_db/)	JPEG images served as static files via FastAPI StaticFiles
Scientific Computing	NumPy 1.26 + SciPy 1.13	Embedding arithmetic, cosine distance calculation
HR Authentication	Django Auth + is_staff	Session-based login, @login_required decorator
Server	Uvicorn (backend), Django dev server (frontend)	Port 8001 backend, 8000 frontend

3. System Architecture

3.1 High-Level Architecture

The system follows a decoupled two-tier architecture. The FastAPI backend is the AI and data engine. The Django frontend is a thin rendering layer. The browser communicates directly with the FastAPI API using JavaScript Fetch, bypassing Django for all data operations. Django only renders HTML pages and enforces authentication.

Tier	Component	Responsibility
Browser	Vanilla JS in Django templates	Camera capture, UI rendering, API fetch calls

Frontend	Django 5.0 (port 8000)	HTML rendering, HR login/logout, CSRF, session
Backend	FastAPI 2.0 (port 8001)	REST API, face recognition, attendance logic, DB
AI Layer	DeepFace + MTCNN + OpenCV	Face detection, embedding extraction, matching
Storage	SQLite x2 + face_db/ folder	attendance.db, hr_auth.db, JPEG face images

3.2 Communication Flow

The browser captures a webcam frame using the HTML5 Canvas API, encodes it as Base64 JPEG, and POSTs it to the FastAPI /api/recognize endpoint. FastAPI decodes the image, runs liveness checks, extracts the Facenet embedding, compares it against all registered employees using cosine distance, and returns a JSON result. The browser renders the result as a toast notification and updates the attendance table.

3.3 Data Storage Strategy

attendance.db (FastAPI)	Stores Employee records (name, ID, embedding JSON, face image path) and Attendance records (date, time, status, confidence). Managed by SQLAlchemy, auto-created on startup.
hr_auth.db (Django)	Stores Django auth users (HR accounts only). Created by Django migrate. Separate from attendance data intentionally — isolation prevents auth bugs affecting operational data.
face_db/ directory	JPEG face images stored per employee as {employee_id}.jpg. Served as static files by FastAPI at /face_images/<filename>. Auto-created on startup.
In-memory liveness state	Per-session frame brightness history (_liveness_history dict) stored in the FastAPI process. Cleared after successful attendance marking.

4. Project Directory Structure

Path	Description
attendance_fixed/	Project root
backend/	FastAPI backend service
main.py	Application entry point — creates tables, registers routers, adds CORS
models.py	SQLAlchemy ORM — Employee and Attendance table definitions
schemas.py	Pydantic v2 schemas — request/response type validation
database.py	SQLite engine, session factory, get_db() dependency
routers/students.py	Employee CRUD: list, get, register (multipart), delete
routers/recognition.py	POST /api/recognize — liveness + embedding + match + mark
routers/attendance.py	GET today stats, dashboard stats, list records, delete record
services/face_service.py	Core AI engine — embedding, liveness, matching, duplicate check

face_db/	Stores JPEG face images, served as static files
requirements.txt	Backend Python dependencies
frontend/	Django frontend service
manage.py	Django management entry point
setup_hr_user.py	One-time script to create hr_admin superuser
attendance_frontend/settings.py	Django settings — FASTAPI_BASE_URL, HR credentials, DBs
attendance_frontend/urls.py	Root URL config, delegates to app/urls.py
app/views.py	Page views — dashboard, attendance, register, employees, reports, hr_login
app/urls.py	URL patterns for all pages + hr-login + hr-logout
app/templates/app/base.html	Sidebar, topbar, apiFetch() JS helper, toast notification system
app/templates/app/dashboard.html	Live stats cards + today's attendance table
app/templates/app/attendance.html	Webcam live feed + recognition result panel
app/templates/app/register.html	3-step employee registration form with webcam capture
app/templates/app/students.html	Employee cards grid with photo and delete action
app/templates/app/reports.html	Filterable, sortable attendance table with CSV export
app/templates/app/hr_login.html	HR-only login page with CSRF token
start_all.bat	Starts both servers + opens browser (Windows)
start_backend.bat	Starts FastAPI backend only
start_frontend.bat	Starts Django frontend only
install_requirements.bat	Installs all pip dependencies once
README.md	User-facing setup and usage documentation
FIXES.md	Detailed changelog of all bug fixes and new features

5. Database Design & Data Models

5.1 Employee Table (attendance.db)

Table Name	employees
Primary Key	id (Integer, auto-increment)
employee_id	VARCHAR(50), unique, indexed — the human-readable staff ID
name	VARCHAR(100), NOT NULL
email	VARCHAR(100), nullable
department	VARCHAR(100), nullable
designation	VARCHAR(100), nullable
face_embedding	TEXT — JSON-serialised Facenet embedding. Single embedding is a flat list of 128 floats. Multi-sample registration stores a JSON array of arrays; the service averages them at match time.
face_image_path	VARCHAR(255) — relative filename (e.g. EMP001.jpg) served from /face_images/
created_at	DATETIME — server default = current timestamp

5.2 Attendance Table (attendance.db)

Table Name	attendance
Primary Key	id (Integer, auto-increment)
Unique Constraint	uq_attendance_per_day — (employee_db_id, date) — one record per person per day
employee_db_id	Integer, FK to employees.id (not enforced at DB level)

employee_name	VARCHAR(100) — denormalised for query simplicity
employee_id	VARCHAR(50) — denormalised staff ID
date	DATE, indexed — the attendance date
time	VARCHAR(20) — HH:MM:SS string
status	VARCHAR(20), default 'present'
confidence	FLOAT — match confidence 0–100%
marked_at	DATETIME — server default = current timestamp

5.3 HR Auth Database (hr_auth.db)

Django's standard auth tables are created by python manage.py migrate --run-syncdb. The setup_hr_user.py script creates a single superuser (is_staff=True, is_superuser=True). The @login_required decorator on the reports view enforces HR authentication; regular non-staff accounts are explicitly denied even with valid credentials.

6. REST API Reference

The full interactive API is available at http://localhost:8001/docs (Swagger UI) and http://localhost:8001/redoc when the backend is running. All endpoints are prefixed with /api/.

6.1 Health

GET /	Returns application name, version, and docs URL
GET /health	Returns {"status": "ok"} — used by the sidebar status indicator

6.2 Employee Endpoints

Method	Endpoint	Description
GET	/api/employees/	List all employees ordered by name. Returns EmployeeResponse[]
GET	/api/employees/stats	Returns {total_employees: int}
GET	/api/employees/{id}	Get single employee by database ID
POST	/api/employees/register	Register new employee. Multipart form: name, employee_id, email?, department?, designation?, image (file), extra_images? (JSON array of base64 strings). Runs duplicate ID check, face extraction, duplicate face check, then persists.
DELETE	/api/employees/{id}	Delete employee + remove face image from disk

6.3 Recognition Endpoint

POST /api/recognize	Core attendance marking endpoint. Accepts JSON body with image (base64 data URL) and session_key (string). Runs liveness check, face extraction, employee matching. Returns RecognitionResult with: recognized, employee_name, employee_id, confidence, attendance_marked, already_marked, message.

6.4 Attendance Endpoints

Method	Endpoint	Description
GET	/api/attendance/today	Today's records + stats (total, present, absent). Returns TodayStats.
GET	/api/attendance/stats	Dashboard stats: today_present, week_total, total_employees, today_absent
GET	/api/attendance/	Filtered records. Query params: date (YYYY-MM-DD), employee_id. Max 500 results.
DELETE	/api/attendance/{id}	Delete a single attendance record

7. AI & Face Recognition Engine

7.1 Model Overview

The face recognition engine is built on DeepFace 0.0.91 using the FaceNet model. FaceNet maps a face image to a 128-dimensional embedding vector in a Euclidean space such that distances between embeddings reflect face similarity. The system uses cosine distance (not Euclidean) for more robust matching across varying lighting and camera conditions.

Model	FaceNet (Facenet512 variant via DeepFace)
Embedding Dimensions	128 floats per face
Distance Metric	Cosine distance (SciPy implementation)
Recognition Threshold	COSINE_THRESHOLD = 0.45 (raised from 0.40 for mobile compatibility)
Min Confidence	MIN_CONFIDENCE_PCT = 55.0% — below this, prompts user to scan again
Duplicate Threshold	DUPLICATE_FACE_THRESHOLD = 0.40 — kept strict for registration
Confidence Formula	confidence = round((1.0 - cosine_distance) * 100, 2)

7.2 Face Detection Strategy (5-Tier Fallback)

To handle diverse image quality, the system tries detectors in order until one succeeds:

Tier 1	OpenCV detector, enforce_detection=True
Tier 2	MTCNN detector, enforce_detection=True
Tier 3	OpenCV detector, enforce_detection=False
Tier 4	MTCNN detector, enforce_detection=False
Tier 5	Skip detector (whole-frame fallback), enforce_detection=False

7.3 Image Preprocessing Pipeline

Before embedding extraction, every image passes through preprocess_image():

- Upscale: images smaller than 400px on the longest edge are scaled up (bicubic interpolation) to improve detection accuracy
- Downscale: images larger than 1280px are scaled down to reduce memory and inference time

- CLAHE (Contrast Limited Adaptive Histogram Equalisation): applied in LAB colour space on the L channel to normalise brightness and improve recognition in dim webcam conditions

7.4 Anti-Spoofing / Liveness Detection

Two independent methods must both pass for attendance to be marked:

Method	How It Works	What It Blocks
Texture Analysis	Laplacian variance of the grayscale frame. Real skin has organic texture. Thresholds: Min=30, Max=6000.	Phone screens (score <30), heavily printed photos with extreme sharpness (score >6000)
Frame Variation	Standard deviation of brightness across last 3–10 frames per session. Threshold: variation < 0.3 = reject.	Static phone/photo held still — a live person breathing produces measurable frame variation

A per-session brightness history dictionary (_liveness_history) tracks frame values in memory. The session_key from the client (typically a tab/device identifier) scopes the history. History is cleared after a successful attendance mark.

7.5 Multi-Sample Registration

During registration, the user can submit additional webcam samples alongside the primary photo (via the extra_images JSON array field). Up to 4 extra samples are accepted. All valid embeddings are stored as a JSON array. At match time, the stored embeddings are averaged (np.mean) into a single representative vector for comparison. This improves robustness across different angles and lighting conditions.

7.6 Duplicate Face Detection

Before completing registration, the new embedding is compared against every existing employee using the same cosine distance function. If the nearest match falls below DUPLICATE_FACE_THRESHOLD (0.40), registration is rejected with a clear error identifying the already-registered employee. This prevents a person from registering twice under different employee IDs.

8. Frontend Architecture

8.1 Django Configuration

The Django frontend is configured as a minimal rendering layer. Key settings:

TIME_ZONE	Asia/Kolkata — all times displayed in IST
FASTAPI_BASE_URL	http://localhost:8001 (env override via FASTAPI_BASE_URL variable). Injected into every template via context processor, consumed by JS as API_BASE.
LOGIN_URL	/hr-login/ — Django's @login_required redirects here
Databases	Two: hr_auth.db (Django auth) and attendance.db (FastAPI, not touched by Django)
ALLOWED_HOSTS	* — development mode

DEBUG	True — development mode; change for production
SECRET_KEY	Hardcoded dev key — must be changed for any production deployment

8.2 URL Routing

/ (dashboard)	views.dashboard — renders dashboard.html
/attendance/	views.attendance — renders attendance.html with live camera
/register/	views.register — renders 3-step registration form
/employees/	views.employees — renders employee card grid
/reports/	views.reports — @login_required, renders reports table
/hr-login/	views.hr_login — GET shows form, POST authenticates + checks is_staff
/hr-logout/	views.hr_logout — logs out, redirects to dashboard

8.3 Template Architecture

All pages extend base.html which provides the sidebar navigation, top bar, apiFetch() JavaScript helper function, and a toast notification system. The API_BASE URL is injected into every template as a context variable and picked up by JavaScript for all fetch calls.

- base.html: Sidebar shows lock icon on Reports link for non-HR users. Footer toggles HR Login / HR Logout based on session state.
- dashboard.html: Calls /api/attendance/stats and /api/attendance/today on load. Displays today present/absent counts and a table of today's attendance records.
- attendance.html: Accesses the webcam via getUserMedia(). Captures frames using HTML5 Canvas, converts to Base64 JPEG, POSTs to /api/recognize every ~1.5s during scanning. Displays recognition result with employee name and confidence.
- register.html: 3-step form. Step 1: employee details. Step 2: photo upload or webcam capture with optional multi-shot. Step 3: confirmation. POSTs to /api/employees/register as multipart/form-data.
- students.html: Loads employee list from /api/employees/ and renders as a card grid with face photo and delete button.
- reports.html: Loads attendance records with date/employee_id filter params. Supports CSV export. HR login required.
- hr_login.html: CSRF-protected login form with password show/hide toggle. Styled to match the dark theme.

9. Security Architecture

9.1 HR Access Control

The reports page is protected by Django's @login_required decorator. Authentication uses Django's built-in session framework. Only users with is_staff=True or is_superuser=True are granted access — regular accounts are explicitly denied even with valid credentials. Sessions are stored in hr_auth.db. CSRF protection is enforced on all POST forms via Django's CsrfViewMiddleware.

9.2 Anti-Spoofing Layers

Layer 1 — Texture	Laplacian variance check (30–6000). Screens and printed photos have abnormally low or high texture scores.
Layer 2 — Motion	Frame variation standard deviation check (>0.3 required). Static images have near-zero variation across frames.
Both Required	Both checks must pass independently — passing one and failing the other results in rejection.
Threshold Tuning	Thresholds calibrated for real mobile cameras: TEXTURE_MIN lowered from 100 to 30, variation threshold lowered from 0.8 to 0.3, TEXTURE_MAX raised from 4500 to 6000.

9.3 Duplicate Registration Prevention

Before persisting a new employee, the system extracts a Facenet embedding from the registration photo and computes cosine distance against every existing employee's stored embedding. Distance below 0.40 = duplicate face detected. Registration is rejected with the name of the already-registered matching employee.

9.4 Recognition Confidence Threshold

Matches are only accepted if the cosine distance is below COSINE_THRESHOLD (0.45) AND confidence is at or above MIN_CONFIDENCE_PCT (55%). Near-misses (distance below threshold but confidence below 55%) return a 'scan again' prompt rather than marking attendance. This avoids false negatives from poor lighting or partial face occlusion.

9.5 One Attendance Per Day

A UNIQUE constraint on (employee_db_id, date) at the database level prevents duplicate attendance records. The recognition endpoint checks for an existing record before inserting and returns already_marked=True with no DB write if a record already exists for today.

9.6 Production Security Notes

- SECRET_KEY must be replaced with a strong random key before any production deployment
- DEBUG=True must be changed to DEBUG=False in production
- ALLOWED_HOSTS should be restricted to known hostnames/IPs
- HR credentials (HR_USERNAME, HR_PASSWORD) should be set via environment variables, not hardcoded
- CORS allow_origins=['*'] in FastAPI should be narrowed to the specific frontend origin
- HTTPS must be enabled for any deployment accessible beyond localhost

10. Key Implementation Details

10.1 Embedding Storage Format

Single-sample registration: face_embedding stores a flat JSON array of 128 floats, e.g. [0.12, -0.45, ...]. Multi-sample registration: face_embedding stores a JSON array of arrays, e.g. [[...], [...], [...]]. The

_get_stored_array() function detects the format by checking if data[0] is a list and averages multi-sample arrays using np.mean(..., axis=0).

10.2 DeepFace API Compatibility

The _represent() helper uses Python's inspect module to detect whether the installed DeepFace version accepts the parameter as 'img' or 'img_path'. It tries the numpy array path first (faster, no disk I/O), then falls back to writing a temporary JPEG to disk (maximum compatibility). The temp file is always cleaned up in a finally block.

10.3 Face Image Path Portability

Face images are stored as relative filenames (e.g. EMP001.jpg) in the face_image_path column, not absolute paths. The delete endpoint resolves the filename against FACE_DB_DIR. This makes the project portable — moving the directory to another machine does not break image references.

10.4 Session Key for Liveness

The browser sends a session_key string with each recognition request. This scopes the per-session frame brightness history so that multiple browser tabs or devices can run liveness checks simultaneously without interfering. After a successful attendance mark, reset_liveness_session() clears the history for that session_key.

10.5 Pydantic v2 Compatibility

The attendance router uses model_validate() (Pydantic v2 replacement for from_orm()) to serialise SQLAlchemy objects. All schemas include class Config: from_attributes = True to enable ORM mode.

10.6 CORS Configuration

FastAPI allows origins http://localhost:8000 and http://127.0.0.1:8000 explicitly, plus '*' as a permissive fallback for development. allow_credentials=True is required to support session cookies if any cross-origin auth is needed.

10.7 Django Auth Integration Fixes

The original project was missing django.contrib.auth, django.contrib.sessions, django.contrib.messages, django.contrib.contenttypes, and their corresponding middleware. Without these, sessions and authentication silently failed. DATABASES was also an empty dict, which caused migrate to fail. The _ctx() helper was fixed to accept request as its first argument so is_authenticated is available in templates.

11. Setup & Deployment

11.1 Prerequisites

Python	3.9 – 3.11 (3.10 recommended)
OS	Windows (bat scripts provided), Linux/Mac (manual commands)
Hardware	Webcam connected to the machine
Disk Space	~2 GB (DeepFace + TensorFlow model weights)
Internet	Required on first run only — DeepFace auto-downloads Facenet model (~90 MB)
Node.js	Not required (no Node.js in the project)

11.2 Windows Quick Start

- Step 1: Double-click install_requirements.bat — installs all backend and frontend Python packages. Takes 3–10 min on first run.
- Step 2: Double-click start_all.bat — opens two terminal windows (backend + frontend) and launches the browser automatically.
- Step 3: Navigate to http://localhost:8000 to use the system.

11.3 Manual Start (Linux / Mac)

Terminal 1 — Backend:

```
cd backend
pip install -r requirements.txt
uvicorn main:app --host 0.0.0.0 --port 8001 --reload
```

Terminal 2 — Frontend:

```
cd frontend
pip install -r requirements.txt
python manage.py migrate --run-syncdb
python setup_hr_user.py
python manage.py runserver 0.0.0.0:8000
```

11.4 Application URLs

http://localhost:8000/	Dashboard — live stats and today's attendance
http://localhost:8000/register/	Register new employee
http://localhost:8000/attendance/	Mark attendance via webcam
http://localhost:8000/employees/	View and manage all employees
http://localhost:8000/reports/	HR-only attendance reports (login required)
http://localhost:8001/docs	FastAPI Swagger interactive API documentation
http://localhost:8001/redoc	FastAPI ReDoc API documentation

11.5 Default HR Credentials

Username	hr_admin
Password	HRSecure@2024
Change via	Set HR_USERNAME and HR_PASSWORD environment variables before starting

12. Troubleshooting Guide

Problem	Solution
API Offline (red dot in sidebar)	Ensure start_backend.bat is running and port 8001 is free. Check Task Manager for port conflicts.
Face not detected	Use good lighting, face camera directly, remove glasses. Ensure face fills at least 30% of the frame.
Attendance not marking	Confidence below 55%. Ensure face is centred, well-lit, and unobstructed. Try registering with additional webcam samples.
Spoofing detected — texture too low	Do not show a phone screen or printed photo. Use a live camera. Mobile cameras on poor-quality streams may trigger this; try increasing lighting.
Spoofing detected — no movement	The frame variation check requires slight natural movement. Ensure the live feed is not frozen or extremely stable.
Duplicate face error on registration	That face is already registered under another employee ID. Each person can only be registered once.
Duplicate employee ID error	That employee ID string already exists in the database. Choose a different ID.
DeepFace download hang	Allow internet access on first run. The Facenet model (~90 MB) downloads once to ~/.deepface/ automatically.
No module named 'tf_keras'	Run: pip install tf-keras in the backend directory. Required for TF 2.16+ compatibility.
Port already in use	Change --port 8001 or --port 8000 in the bat files. Update FASTAPI_BASE_URL in settings.py if the backend port changes.
HR login not working	Run: cd frontend && python setup_hr_user.py to create or reset the hr_admin superuser.
manage.py migrate fails	Ensure Python can write to the frontend directory. Delete hr_auth.db and re-run migrate if the schema is corrupted.
Camera permission denied in browser	Allow camera access in browser settings. Chrome/Edge require HTTPS or localhost for getUserMedia().
Low match confidence after registration	Re-register with multiple webcam shots (multi-sample). Ensure the registration photo is well-lit, front-facing, and unobstructed.

13. Bug Fixes & Changes Applied

13.1 Django App Startup Fixes

- Missing INSTALLED_APPS: django.contrib.auth, django.contrib.sessions, django.contrib.messages, django.contrib.contenttypes were absent — Django could not run sessions or authentication at all.
- Empty DATABASES: DATABASES = {} meant Django had no database, causing migrate to fail silently and sessions to not persist.
- Missing CSRF Middleware: CsrfViewMiddleware was not in the MIDDLEWARE list, causing all POST forms to return 403 Forbidden.
- _ctx() helper: Missing request parameter meant request.user was unavailable in templates; fixed to def _ctx(request, extra=None) and added is_hr flag.

13.2 Face Recognition Improvements

- 5-tier face detector fallback strategy (opencv → mtcnn → relaxed modes → skip detector)
- CLAHE image preprocessing added for better webcam performance in dim lighting
- Cosine threshold raised from 0.40 to 0.45 for better mobile photo matching
- Min confidence lowered from 60% to 55% to reduce false rejections
- Camera warm-up delay added before scanning starts
- Mirror transform removed from canvas capture to prevent embedding mismatch
- Multi-sample (1–5) face registration with averaged embeddings
- Liveness texture minimum lowered from 100 to 30 for mobile camera compatibility
- Liveness variation threshold lowered from 0.8 to 0.3 for stabilised cameras

13.3 New Feature: HR Authentication

- @login_required decorator added to the reports view
- hr_login view: handles GET (show form) and POST (authenticate + check is_staff)
- hr_logout view: logs out, redirects to dashboard
- LOGIN_URL = '/hr-login/' added to settings.py
- HR credential settings (HR_USERNAME, HR_PASSWORD) with env variable override
- New hr_login.html template with CSRF protection, dark theme, password show/hide toggle
- Reports nav item in base.html shows lock icon for non-HR users
- setup_hr_user.py script to create/reset the HR superuser

14. Dependency Reference

14.1 Backend Dependencies

Package	Version	Purpose
fastapi	0.110.0	Async REST API framework with automatic OpenAPI docs
uvicorn[standard]	0.29.0	ASGI server — runs the FastAPI application
python-multipart	0.0.9	Multipart form parsing for file uploads
sqlalchemy	2.0.29	ORM for database models and session management
pydantic	2.6.4	Data validation and serialisation for API schemas
deepface	0.0.91	Facial recognition library wrapping Facenet, VGG-Face, etc.
opencv-python	4.9.0.80	Image decoding, preprocessing, CLAHE, face detection fallback
numpy	1.26.4	Embedding arithmetic, array operations
pillow	10.3.0	Image handling library (DeepFace dependency)
scipy	1.13.0	Cosine distance calculation for embedding comparison
mtcnn	0.1.1	Multi-task Cascaded CNN face detector (primary detector)
tf-keras	>=2.16.0	Keras 2 shim for TF 2.16+ compatibility with DeepFace

aiofiles	23.2.1	Async file I/O utilities

14.2 Frontend Dependencies

Package	Version	Purpose
django	5.0.4	Web framework — template rendering, auth, sessions, CSRF
requests	2.31.0	HTTP client (available in frontend; API calls are done by JS Fetch)
python-decouple	3.8	Environment variable configuration management

15. References & Acknowledgements

DeepFace	Open-source facial attribute analysis library by Sefik Ilkin Serengil. GitHub: github.com/serengil/deepface
FaceNet	Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. arXiv:1503.03832. Google Inc.
FastAPI	Modern, fast web framework for building APIs with Python 3.7+ by Sebastián Ramírez. fastapi.tiangolo.com
MTCNN	Multi-task Cascaded Convolutional Networks for face detection. Zhang et al. (2016).
Django	High-level Python web framework. djangoproject.com
SQLAlchemy	The Python SQL toolkit and ORM. sqlalchemy.org
OpenCV	Open Source Computer Vision Library. opencv.org
HCL Technologies	Internship project host organisation

MIT License — free to use, modify, and distribute.